Microservices in Agile Software Development: a Workshop-Based Study into Issues, Advantages, and Disadvantages

Davide Taibi, Valentina Lenarduzzi, Claus Pahl, Andrea Janes
Free University of Bolzano-Bozen
Bolzano-Bozen, Italy
davide.taibi; valentina.lenarduzzi; claus.pahl; andrea.janes@unibz.it

ABSTRACT

In the last years, cloud-native architectures have emerged as a target platform for the deployment of microservice architectures. The migration of existing monoliths into cloud-native applications is still in the early phase, and only few companies already started their migrations. Therefore, success and failure stories about different approaches are not available in the literature. This context connects also to the recently discussed DevOps context where development and continuous deployment are closely linked.

KEYWORDS

Cloud Software, Software Architecture, Microservices, SOA

1 INTRODUCTION

Software architectures for large and complex systems have been a topic for research since several years. This can be also observed looking at how companies organize their teams, source code versioning systems, and packages. Monolithic systems are often developed, deployed and tested as one big self-standing piece of code. The modification of one feature in a monolithic system often requires the execution of comprehensive test suites to test the whole system. Experience teaches that monoliths often turn into unmaintainable systems.

Starting from 2012 [2], Microservices have become an important architectural style, thanks to their decomposable nature. Microservices allow developers to decompose products into building blocks that do only one function, and do it well [3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

XP '17 Workshops, May 22-26, 2017, Cologne, Germany
© 2017 Association for Computing Machinery.
ACM ISBN 978-1-4503-5264-2/17/05...$15.00
https://doi.org/10.1145/3120459.3120483

Moreover, microservices have a set of different properties, since they need to be autonomous, isolated, elastic, resilient, and responsive. These properties differentiate them from previous service-oriented approaches. Microservice-based architectures are complex to design, requiring substantial expertise. Companies that decide to adopt microservice-based architectural styles, need to adapt their standards and practices, by adding an extra overhead to the design phase. As any new technology, microservices require training and effort overhead for the migration to the new patterns.

On one hand, microservices can help developers to split the development into autonomous tasks, improving user story assignment to developers, on the other hand, they require to implement a complex architecture and a messaging system to communicate between each other that require a lot of planning effort that contradicts agility.

Our goal is to identify issues, advantages and disadvantages in migrating monolithic application into microservices-based in agile context. In order to better identify the current uncertainties, we organized a community workshop.

This paper presents the methodology and the results of that workshop, which was held during the 2017 International Conference on Agile Processes in Software Engineering and Extreme Programming. The workshop aimed at collecting experiences on microservice adoption, reporting best practices, but specially failure cases, so as to build a community knowledge based on previous errors and successes.

The remainder of the paper is organized as follows. Section 2 provides an overview on microservices, Section 3 describes the workshop methodology in detail. Section 4 reports the workshop results, and Section 5 discusses the implications and limitations of the results. Finally, Section 6 concludes the paper and highlights future work.

2 Introduction to Microservices

Monolithic cloud-applications are usually deployed as a single package on a web container such as Tomcat or JBoss. Such systems are easier to deploy and to develop, specially in case of non-experienced developers. However, monolithic systems do not easily scale and tend to become complex to understand and to maintain in the long term. Monolithic architectures are

commonly not suitable to support the high requirement volatility and speed of delivery required in the last years. To overcome to his issue, new architectural styles have been proposed in the last years.

Service Oriented Architectures (SOA) tried to solve the problem of complexity dividing applications into sets of business applications offering services through different protocols. Communication among different business applications are usually delegated to an Enterprise Service Bus (ESB) that route the messages among them. However, the development of SOA applications is very expensive and complex, since they are designed to support very high workloads, with a huge number of users. Moreover, the ESB was not designed for the cloud and, adding and removing new servers on demand is complicated. In order to address the problems of the monolithic applications, and to take advantages of the SOA architectures.

A lightweight SOA approach, similar to microservices, was described informally by Jim Gray in ACM queue in 2006 [1]. The term Microservice has been widely used from March 2012, after a technical report [2] published by ToughtWorks. Microservices were then introduced in two technical workshops in 2012 by James Lewis [3] (March 2012) and by Fred George [4] (September 2012) and then described more in detail by Lewis and Fowler in 2014 on Fowler's blog [10]. Nowadays, the microservice architectural style is largely used by several big and small companies such as Amazon [5], Netflix [6], Linkedin [7], SoundCloud [8] and many others.

Microservices are relatively small and autonomous services deployed independently, with a single and clearly-defined purpose. Because of their independent deployment, they have a lot of advantages. They can be developed in different programming languages, they can scale independently from other services and they can be deployed on the hardware that best suit their needs. Moreover, because of their size, they are easier to maintain and more fault tolerant since the failure of one service will not break the whole system as it could happen in a monolithic system. Another characteristic of microservices is, being cloud native applications, the support of the IDEAL properties: Isolation of state, Distribution, Elasticity, Automated management and Loose Coupling [9]. Microservices proposes to vertically decompose the applications into a subset of business-driven services. Every service can be developed, deployed and tested independently by different development teams, and by means of different technology stacks. The responsibility of the development of a microservice belong only to one team, who is in charge of the whole development process, including deploying, operating and upgrading the service when needed.

3 METHODOLOGY

This section describes the methodology we followed when planning and executing the community workshop.

The primary goal of the workshop was to collect experiences on microservice adoption, reporting best practices, and problems in migrating from monolithic architectures to microservices in the context of agile development processes.

To gather this input, we conducted a three-hour workshop during the 2017 International Conference on Agile Processes in Software Engineering and Extreme Programming, an international conference mainly targeted at practitioners developing with agile processes. We chose this venue due to the high presence of practitioners with experience both in microservices and in agile software development.

Consistent with the goal of the workshop, participants discussed:

- Barriers to migrating to microservices from a lack of adequate techniques or tooling
- Major issues during migration to microservices
- Migration issues related to the agile process
- Benefits of migrations

3.2 Participants

The participant population included five industry practitioners from different companies, two doctoral students. Both doctoral students were from the computer science or software engineering discipline. The group included representatives from Germany (2) Italy (3), India (2), Czech Republic (1). All participants, except the Ph.D. student have experience in developing microservices-based systems. The first author of this paper served as workshop facilitator.

3.3 Data Collection Process

We employed the Nominal Group Technique [22] for data collection during the workshop because of its usefulness in the type of context present in the workshop [12], namely:

- Most participants are new to one another (i.e. they do not generally work with one another).
- We were not aware a priori of particular issues that could have been controversial or generated heated interactions.

3.3.1 Introduction and explanation phase

After a short introduction of each participant, the moderator provided a brief overview of microservices to ensure that everyone was using the same terminology and then described the study goal and process.

3.3.2 Idea generation phase

The first step was a type of brainstorming in which the participants were encouraged to write down as many issues, advantages and disadvantages for the migration of monolithic systems to microservices-based ones. Participants performed this task individually and in silence to encourage participation from all attendees and to prevent some of the more vocal participants from dominating the discussion. For each idea identified by a participant, he or she created a Post-it note that contained the name of the issues, advantages and disadvantages. The idea generation phase lasted 10 minutes.

3.3.3 Idea sharing and clustering

Each participant described their notes on issues, advantages and disadvantages and we created an initial set of categories around which to organize the discussion.

Each participant clustered similar issues, advantages and disadvantages. This activity continued for 30 minutes, during which time the facilitators were available to answer questions or provide clarification. Figure 1 shows some of the activity during this phase.

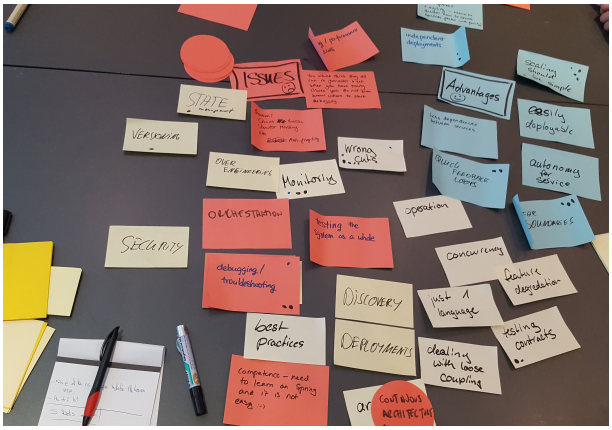

Figure 1: The Idea Sharing Phase

3.3.5 Voting and ranking phase

In order to capture the importance of each issues, advantages and disadvantages, the participants voted. We gave each participant 5 paste-on dots for each group (5 for issues, 5 for advantages and 5 for disadvantages). We asked the participants to consider all of the identified issues, advantages and disadvantages (from all discussion categories) and to determine which ones they thought most important (i.e., in need of being addressed with the highest priority by the research community). Participants were free to stick their dots as they like, with the limit of a maximum of five dots per group.

4. RESULTS

In this section, we report the results obtained in the community workshop.

4.1 Idea Generation and Idea Sharing Phases

During the Idea Generation phase, the workshop participants individually recorded 29 issues, 10 advantages and 7 disadvantages. Subsequently, during the Idea Sharing phase, the participants decreased this number removing duplicates and resulting in 7 issues, 4 advantages and 3 disadvantages.

There was one other important result of the Idea Sharing Phase.

4.3 Voting and Raking Phase

After completion of the Group Discussion phase, each group presented the results of their deliberation by briefly explaining each issue, advantage, disadvantage. The last column of Table 1, Table 2 and Table 3 show the number of votes received.

As for the issues in migrating to microservices, the workshop participants cast more votes (40% of the total cast votes) for the wrong cuts when splitting the monolith to several microservices. Moreover, continuous architecture is also scored as a relevant issue (26% of total votes).

Taking into account the advantages of the migration, the increase and the easiness of scalability is by far the most voted one, collecting more than 41% of preferences while the easiness of the identification of clear boundaries is also considered important (25%).

Considering the Disadvantages, both the need of senior developers and the learn difficulty are considered the main barriers for the migration, collecting 46% of preferences each.

Table 1, Table 2 and Table 3 report the list of issues, advantages and disadvantages together with their votes and description as reported during the workshop.

5. DISCUSSION

The results of this workshop provided additional evidence about previously identified issues, advantages and disadvantages of microservices.

As for the main issues identified, the system decomposition is confirmed to be the most important issue. Decomposing the system is generally complex, and this is also true when decomposing the system in sub-requirements in agile software development [18]. Therefore, easier and more clear decomposition strategies should be addressed by the research community. Moreover, the continuous architecture approach is still not easily applicable probably due to the lack of communication, or the lack of documentation on the architectural decision. Agile development processes decrease the time to market while reducing the need of communication between developers [17]. The development of a distributed system, such these based on microservices can benefit of the agile practices, however, developers should take into account the distributed nature of the project and therefore consider to communication strategies to transfer the knowledge between teams. Considering the communication between service, we recommend to keep into account different interoperability solutions [16], specially when dealing with the communication between services. Monitoring microservices is a complex task, due to the distributed nature of them. Monitoring systems and dynamic measures such as [11][20] and [21] could be highly beneficial approaches to monitor them. Participants also highlighted the issue of estimating the effort of the whole system could be trivial, while the effort estimation of the single microservice is much easier than estimating the development of the same feature into a monolith. Functional size measures could be a possible research direction to support practitioners in

estimating the whole system effort, as for example in [19]. Last, but not least, beside agile software development – and specially SCRUM [14] - can be easily introduced into waterfall or ad-hoc development processes, we recommend to start the migration to microservices by using a development process already known by the development team and not starting the migration to microservices while changing the process from an ad-hoc process to agile.

Taking into account the main benefit, the increase in scalability and the clear boundaries are highly appreciated benefits for the migration. However, participants also confirmed that, as reported into the issues, the decomposition of the system into microservices must be correctly addressed, considering vertical services and starting to the development of the minimum viable product [15].

The main disadvantages reported are related to the difficulty to learn. Junior developers need a long training before adopting microservices, and even senior developers need to be accurately trained more than what is generally needed to develop a monolithic solution. Another potential problem that moderately affected our participants is the possibility of accessing to private information and the possible propagation of unauthorized information, shared between services. Approaches such as [13] could be easily extended by researchers to address this issue.

6 CONCLUSION AND FUTURE WORK

In this paper we described the goals, methodology, and results of a community workshop held during XP 2017 with the goal of identifying issues, advantages and disadvantages of microservices applied in an agile development process. During the workshop, we collected data using the Nominal Group Technique [22], and after the workshop we analyzed the collected data.

The results of the community workshop confirm that microservices, beside than being well known are still complex to adopt and the success of the migration from monolithic systems depends on several factors.

Future works include the analysis of the problems and issues so as to support practitioners in identifying potential solutions.

Table 1: The Issues Identified

Issue	Description	Votes
Wrong Cuts	Decomposing the monolith into microservices is a complex task. Understanding which is the most effective decomposition strategy and where to "cut" the existing system is still not clearly defined.	12
Continuous Architecture	The continuous changes required to the architecture require a lot of attention and often create issues, especially if the architecture is not documented.	8
Monitoring	Monitoring microservices require a distributed monitoring strategy while monoliths are easier to monitor	3
Testing Contracts	Testing the whole system is more complex. Testing interfaces must remain stable, a set of testing contracts should be clearly defined between microservices.	3
Versioning	Microservices versioning often create issues for connected services. As example, API should be deprecated and not simply changed, however, the API versioning is still an issue that would require a simpler mechanism.	2
State Management	Microservices are stateless, however in some cases we do need to keep track of the application state.	2
System Effort Estimation	The effort estimation of the whole system is more complex and less accurate compared to monolithic systems.	

Table 2: The Advantages Identified

Advantage	Description	Votes
Simple Scalability	Because of their nature, microservices are easier to scale than monoliths	10
Clear Boundaries	Clear boundaries reflect clear responsibilities, both for the service point of view (a microservice should do only one thing) and from the development team (the team is clearly responsible of exactly one microservice without having shared responsibilities with other teams.	6
Independent Deployment	Microservices can be deployed independently from the rest of the application while teams working on monoliths need to synchronize to deploy together.	4
Quick Feedback Loops	Independent deployment support fast releases and therefore allowing quick customer feedbacks.	4

Table 3: The Disadvantages Identified

Disadvantage	Description	Votes
Need of senior developers	The architectural decisions are complex and moreover, starting the development of a microservices-based system require highly skilled developers.	12
Difficult to learn	Because of the technologies adopted by microservices (orchestration, distributed system, API management and others) microservices require a lot of training.	12
Increase of system complexity	The spread of microservices and the increase of size of the ecosystem of microservices commonly lead to an increase on the system complexity.	2

REFERENCES

[1] J. Gray, "A conversation with Werner Vogels," http://queue.acm.org/detail.cfm?id=1142065, 2006, accessed: December 2016.

[2] J. Lewis and M. Fowler, "Technology radar," http://www.thoughtworks.com/radar, march 2007, accessed: December 2016.

[3] J. Lewis, "Microservices - java, the unix way," in Proceedings of the 33rd Degree Conference for Java Masters, march 2012.

[4] F. George, "Microservice architecture: a personal journey of discovery," in Barcelona Ruby Conference, ser. BaRuCo '12, September 2012.

[5] S. Kramer, "The biggest thing amazon got right: The platform," https://gigaom.com/2011/10/12/419- the-biggest-thing-amazon-got-right-the-platform/, October 2006, accessed: December 2016.

[6] T. Mauro, "Adopting microservices at netflix: Lessons for architectural design," http://nginx.com/blog/microservices-at-netflix-architectural-best- practices/, February 2015, accessed: December 2016.

[7] S. Ihde, "From a monolith to microservices + rest: the evolution of linkedin's service architecture," http://www.infoq.com/presentations/linkedin-microservices-urn, March 2015, accessed: December 2016.

[8] P. Calcado, "Soundcloud — building products at soundcloud - part i: Dealing with the monolith," https://developers.soundcloud.com/blog/building-products-at-soundcloud-part-1-dealing-with-the-monolith, June 2014, accessed: December 2016.

[9] C. Fehling, F. Leymann, S. T. Ruehl, M. Rudek, and S. Verclas, "Service migration patterns - decision support and best practices for the migration of existing service-based applications to cloud environments," in Proceedings of the 6th IEEE International Conference on Service Oriented Computing and Application (SOCA). IEEE Computer Society, 2013, pp. 9–16.

[10] J. Lewis and M. Fowler, "Microservices," www.martinfowler.com/articles/microservices.html, Accessed: December 2016, March 2014.

[11] D. Tosi, L. Lavazza, S. Morasca, and D. Taibi, "On the Definition of Dynamic Software Measures", in Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), Lund, Sweden, 2012.

[12] T. C. Lethbridge, S. E. Sim, and J. Singer. Studying software engineers: Data collection techniques for software field studies. Empirical Software Engineering, 5(3):311–341, 2000.

[13] B. Carminati, E. Ferrari, S. Morasca, and D. Taibi "A probability-based approach to modeling the risk of unauthorized propagation of information in on-line social networks." 1st ACM Conference on Data and Application Security and Privacy, CODASPY'11; San Antonio, TX; February 2011

[14] L. Lavazza, S. Morasca, D. Taibi, and D. Tosi, "Applying SCRUM in an OSS Development Process: An Empirical Evaluation", in XP 2010 - 11th International Conference in Software Engineering and Extreme Programming, Trondheim, Norway, June 1-4, 2010. Proceedings, 2010, pp. 147-159.

[15] D. Taibi and V. Lenarduzzi, "MVP explained: A Systematic Mapping on the Definition of Minimum Viable Product", Proceedings of the 42th Euromicro Conference Series on Software Engineering and Advanced Applications, SEAA 2016, pp. 112-119 Cyprus, 2016.

[16] H. Abukwaik, D. Rombach, and D. Taibi, "Interoperability-related architectural problems and solutions in information systems: A scoping study", in European Conference on Software Architecture, Vienna, 2014.

[17] D. Taibi, V. Lenarduzzi, M. O. Ahmad, and K. Liukkunen, "Comparing Communication Effort within the Scrum, Scrum with Kanban, XP, and Banana Development Processes" in 21th Evaluation and Assessment in Software Engineeering (EASE), 2017.

[18] D. Taibi, V. Lenarduzzi, A. Janes, K. Liukkunen, and M.O. Ahmad, "Comparing Requirements Decomposition Within the Scrum, Scrum with Kanban, XP, and Banana Development Processes", Agile Processes in Software Engineering and Extreme Programming: 18th International Conference, XP 2017, Cologne, Germany, May 22-26, 2017. pp. 68–83, 2017.

[19] V. Lenarduzzi, I. Lunesu, M. Matta, and D. Taibi, "Functional Size Measures and Effort Estimation in Agile Development: a Replicated Study" Agile Processes in Software Engineering and Extreme Programming: 16th International Conference, XP 2015.

[20] V. Lenarduzzi, A. C. Stan, D. Taibi, D. Tosi, and G. Venters, "A Dynamical Quality Model to Continuously Monitor Software Maintenance", in 11th European Conference on Information Systems Management, 2017.

[21] A. Janes, V. Lenarduzzi, and A. C. Stan. "A Continuous Software Quality Monitoring Approach for Small and Medium Enterprises". 8th ACM/SPEC on International Conference on Performance Engineering Companion (ICPE '17 Companion). ACM, New York, NY, USA, 97-100.

[22] A. Delbecq, A. Van de Ven, and D. Gustafson. "Group techniques for program planning: a guide to nominal group and Delphi processes". Management applications series. Scott, Foresman, 1975.